МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет “Львівська політехніка”

Інститут комп’ютерних наук та інформаційних технологій

Кафедра автоматизованих систем управління

Звіт

про виконання лабораторної роботи №1

з дисципліни “ Паралельні обчислення і розподілені системи”

на тему:

“Способи розпаралелювання та організації обчислень. Послідовні алгоритми”

Виконав:

студент групи ОІ-21СП

Горбатюк Д. О.

Прийняла:

доцентка кафедри АСУ,

Федевич О. Ю.

Львів - 2025

Лабораторна робота №1

Тема: способи розпаралелювання та організації обчислень. Послідовні алгоритми.

Мета роботи: Оволодіти практичними прийомами розробки алгоритмів та програм із застосуванням ітерації.

Завдання лабораторної роботи

Варіант по списку групи - 1

Завдання: Розв’язати систему лінійних алгебраїчних рівнянь за методом Гаусса.

ХІД РОБОТИ

1. Мовою програмування обрано Java, адже ця мова програмування підтримує багатопотоковість і є доволі зручною у використанні. Як середовище розробки обрано Intellij Idea, через зручність створення і підтримки проєктів.
2. Створив проєкт у відповідному середовищі (див. Рис. 1.1):

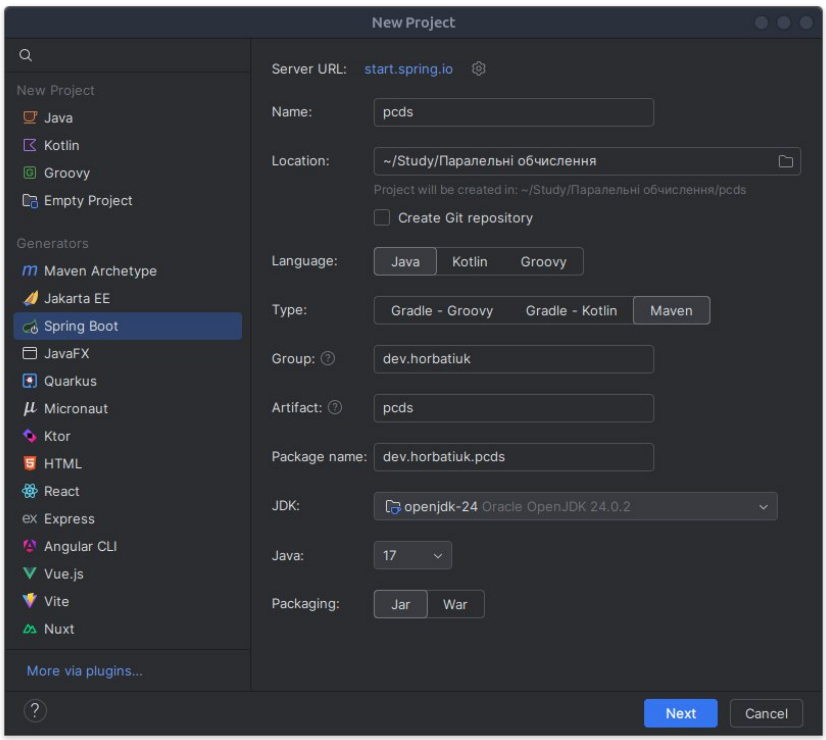
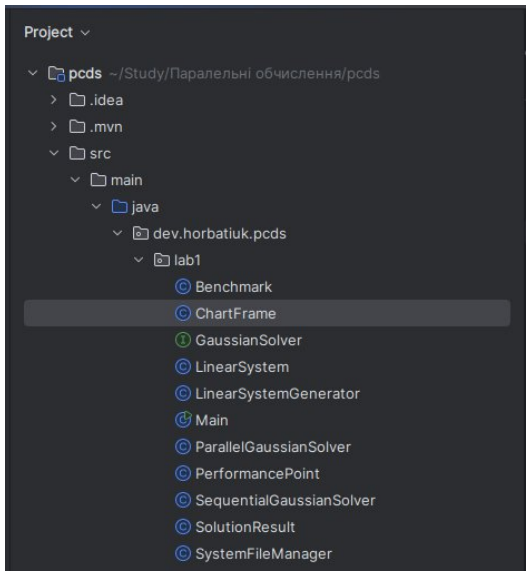


Рис. 1.1. Створення проєкту

3. Створив наступну архітектуру додатку:



4. Написав код для всіх класів (див. Додатки 1-11).
5. Приклад виконання:

Приклад для кількості елементів 999: послідовно і 4ри потоки

```
==================================================
       Gaussian Elimination Laboratory Work
==================================================

==================================================
                      MAIN MENU
==================================================
1 - Generate random system
2 - Read system from file
3 - Use control example
4 - Build speedup chart
0 - Exit
--------------------------------------------------
Choose option: 1

==================================================
             Random system generation
==================================================
Enter system size n: 999
Save generated system to file? (1-yes, 0-no): 0

==================================================
                    Input system
==================================================
System size: 999 x 999

|   5329.000     -7.000      0.000     10.000      2.000     -6.000     -7.000      0.000 ... |      8.000
|      9.000   5190.000      0.000      3.000      8.000      9.000      6.000     -1.000 ... |     -9.000
|     -4.000      2.000   5145.000      4.000     -4.000    -10.000      5.000      8.000 ... |     -9.000
|      6.000     -6.000      1.000   5079.000     -3.000      3.000      8.000     -8.000 ... |      5.000
|     -2.000     -8.000      6.000      0.000   5304.000      8.000      0.000      7.000 ... |     -6.000
```

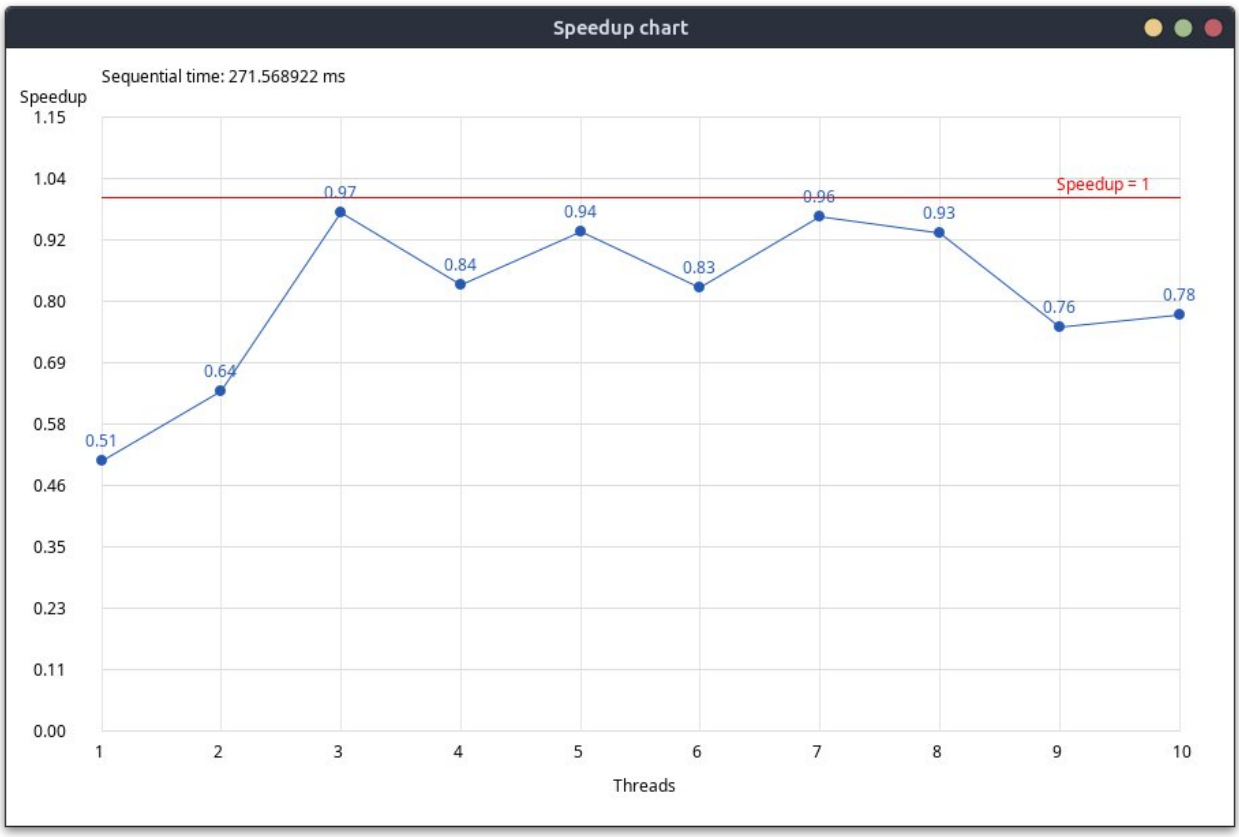
```
|      9.000     -2.000      7.000      7.000      3.000   5127.000      5.000      0.000 ... |     -6.000
|      4.000    -10.000      8.000     -4.000      1.000      2.000   5280.000     -4.000 ... |     -1.000
|     -5.000      6.000     -9.000     -7.000      7.000     -2.000     -2.000   5176.000 ... |      6.000
|       ...                                        |        ...
Enter number of threads: 4

==================================================
                Computation results
==================================================


--------------------------------------------------
SEQUENTIAL SOLUTION
--------------------------------------------------
x[0   ] =     0.001480
x[1   ] =    -0.001725
x[2   ] =    -0.001705
x[3   ] =     0.000970
x[4   ] =    -0.001079
x[5   ] =    -0.001158
x[6   ] =    -0.000240
x[7   ] =     0.001200
x[8   ] =    -0.001100
x[9   ] =     0.000569
...
x[998 ] =    -0.001918
Sequential time: 297.233160 ms

--------------------------------------------------
PARALLEL SOLUTION
--------------------------------------------------
x[0   ] =     0.001480
x[1   ] =    -0.001725
x[2   ] =    -0.001705
x[3   ] =     0.000970
x[4   ] =    -0.001079
x[5   ] =    -0.001158
x[6   ] =    -0.000240
x[7   ] =     0.001200
x[8   ] =    -0.001100
x[9   ] =     0.000569
...
x[998 ] =    -0.001918
Parallel time: 419.200782 ms

--------------------------------------------------
PERFORMANCE SUMMARY
--------------------------------------------------
Sequential time :   297.233160 ms
Parallel time   :   419.200782 ms
Speedup         :        0.7090
Result          : parallel version is slower by 1.41 times
```

Як видно із результатів, у випадку розмірності 999, розділення на 4 потоки є менш ефективним, аніж послідовне виконання.

6. Також, було розроблено модуль, який дозволяє вивести графік ефективності для різної кількості потоків на прикладі однакової розмірності.

Приклад для розмірності СЛАР - 1000: графік ефективності потоків від 1 до 10

```
==================================================
       Gaussian Elimination Laboratory Work
==================================================

==================================================
                      MAIN MENU
==================================================
1 - Generate random system
2 - Read system from file
3 - Use control example
4 - Build speedup chart
0 - Exit
--------------------------------------------------
Choose option: 4

==================================================
              Speedup chart experiment
==================================================
Enter system size n: 999
Enter maximum number of threads: 10

==================================================
                Input system preview
==================================================
System size: 999 x 999

|   5197.000     -3.000      1.000     -2.000      7.000      1.000      0.000     -1.000 ... |      9.000
|     -5.000   5353.000     -1.000     -7.000     10.000     -2.000     -9.000      9.000 ... |     -7.000
|     -4.000     -6.000   5231.000     10.000     -2.000      1.000     -8.000      7.000 ... |      0.000
|     -5.000      1.000      5.000   5039.000     -7.000     -7.000    -10.000      2.000 ... |     -3.000
|      0.000      0.000      6.000      3.000   5327.000     -6.000     -3.000     -3.000 ... |    -10.000
|      8.000      7.000      5.000     -8.000    -10.000   5356.000     -6.000      9.000 ... |      2.000
|      0.000      6.000      1.000      5.000      4.000      3.000   5343.000      1.000 ... |     -1.000
|     -2.000      1.000      5.000     -1.000    -10.000     -9.000      0.000   5214.000 ... |      9.000
|       ...                                     |       ...
Sequential baseline time: 271.568922 ms

Threads | Parallel time (ms) | Speedup
--------------------------------------
      1 |          534.581150 |  0.5080
      2 |          425.598566 |  0.6381
      3 |          279.444674 |  0.9718
      4 |          324.679398 |  0.8364
      5 |          290.280832 |  0.9355
      6 |          326.843200 |  0.8309
      7 |          281.669912 |  0.9641
      8 |          290.956763 |  0.9334
      9 |          358.467141 |  0.7576
     10 |          348.239805 |  0.7798
```



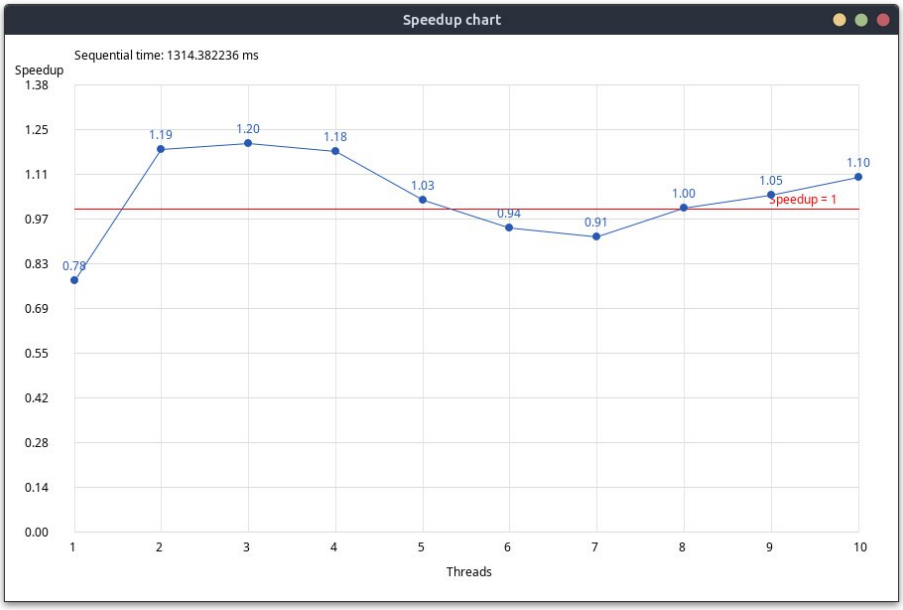
З нашого графіка видно, що швидкість виконання потоково від 2 до 10 не перевершує послідовне виконання. Це зумовлено специфікою самого алгоритму, а також малою розмірністю.

Приклад для розмірності СЛАР - 1500: графік ефективності потоків від 1 до 10

```
==================================================
        Gaussian Elimination Laboratory Work
==================================================


==================================================
                      MAIN MENU
==================================================
1 - Generate random system
2 - Read system from file
3 - Use control example
4 - Build speedup chart
0 - Exit
--------------------------------------------------
Choose option: 4


==================================================
              Speedup chart experiment
==================================================
Enter system size n: 1500
```

```
Enter maximum number of threads: 10


==================================================
                Input system preview
==================================================
System size: 1500 x 1500

|   7624.000      3.000      0.000      9.000     -1.000    -10.000     -5.000      8.000 ... |    -10.000
|      7.000   7992.000      1.000      6.000     10.000     -1.000      9.000      9.000 ... |     -2.000
|    -10.000     -5.000   7815.000     -1.000      1.000     -5.000     -8.000     -3.000 ... |    -10.000
|      2.000    -10.000      8.000   7886.000     -5.000     -2.000      2.000    -10.000 ... |     -8.000
|     -6.000      9.000      1.000      3.000   7691.000     -7.000     -7.000      0.000 ... |      1.000
|     -6.000     -6.000      1.000      8.000      9.000   7873.000      3.000     -7.000 ... |     -6.000
|     -9.000     -7.000      4.000      3.000     -5.000      9.000   7875.000      9.000 ... |      6.000
|     -5.000     -4.000      6.000      7.000      6.000      4.000    -10.000   7694.000 ... |      1.000
|       ...                                        |        ...
Sequential baseline time: 1314.382236 ms
```

Threads	Parallel time (ms)	Speedup
1	1686.578518	0.7793
2	1107.267260	1.1871
3	1092.193203	1.2034
4	1114.789515	1.1790
5	1277.272318	1.0291
6	1395.556751	0.9418
7	1436.590089	0.9149
8	1308.249392	1.0047
9	1257.549150	1.0452
10	1194.663544	1.1002



А з цього прикладу чітко видно, що 2-5 потоків для випадку розмірності 1500 є більш ефективним, аніж послідовне виконання.

ВИСНОВКИ

У ході лабораторної роботи я реалізував програму на Java для розв’язання системи лінійних алгебраїчних рівнянь методом Гаусса та порівняв послідовний і паралельний варіанти її виконання. Також, було організовано генерацію вхідних даних, роботу з файлами та виведення результатів.

За результатами тестування видно, що послідовна реалізація в більшості випадків працює швидше, ніж паралельна, оскільки витрати на створення і синхронізацію потоків виявилися досить великими. Разом з тим, експерименти показали, що зі збільшенням розміру задачі ефективність паралельного підходу зростає.

Також зазначу, що метод Гаусса сам по собі має послідовну структуру, тому повністю ефективно розпаралелити його складно. Кожний наступний крок залежить від результату попереднього, оскільки спочатку потрібно вибрати опорний елемент, виконати перетворення рядків, і лише після цього переходити далі. Через таку залежність між етапами паралельно можна виконувати лише окремі частини обчислень, тому виграш у продуктивності, в основному, є обмежений.

ДОДАТКИ

Додаток 1

Код файлу Benchmark.java

```
package dev.horbatiuk.pcds.lab1;

public class Benchmark {

    private Benchmark() {
    }

    public static SolutionResult measure(GaussianSolver solver, LinearSystem
system) throws Exception {
        LinearSystem copy = system.copy();

        long start = System.nanoTime();
        double[] solution = solver.solve(copy);
        long end = System.nanoTime();

        return new SolutionResult(solution, end - start);
    }

    public static double calculateSpeedup(SolutionResult sequential,
SolutionResult parallel) {
        return sequential.getExecutionTimeNs() / (double)
parallel.getExecutionTimeNs();
    }
}
```

Додаток 2

Код файлу ChartFrame.java

```
package dev.horbatiuk.pcds.lab1;

import javax.swing.*;
import java.awt.*;
import java.util.List;
import java.util.Locale;

public class ChartFrame extends JFrame {

    public ChartFrame(List<PerformancePoint> points, double sequentialTimeMs) {
        setTitle("Speedup chart");
        setSize(900, 600);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        add(new ChartPanel(points, sequentialTimeMs));
    }

    private static class ChartPanel extends JPanel {
        private final List<PerformancePoint> points;
        private final double sequentialTimeMs;

        public ChartPanel(List<PerformancePoint> points, double sequentialTimeMs)
{
            this.points = points;
            this.sequentialTimeMs = sequentialTimeMs;
            setBackground(Color.WHITE);
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);

            if (points == null || points.isEmpty()) {
                g.drawString("No data to display", 50, 50);
                return;
            }

            Graphics2D g2 = (Graphics2D) g;
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            int left = 70;
            int right = 40;
            int top = 50;
            int bottom = 70;
```

```
            int width = getWidth() - left - right;
            int height = getHeight() - top - bottom;

            int x0 = left;
            int y0 = getHeight() - bottom;

            g2.setColor(Color.BLACK);
            g2.drawLine(x0, y0, x0 + width, y0);
            g2.drawLine(x0, y0, x0, top);

            double maxY = 1.0;
            for (PerformancePoint p : points) {
                maxY = Math.max(maxY, p.getSpeedup());
            }
            maxY *= 1.15;

            int maxThreads = points.stream()
                    .mapToInt(PerformancePoint::getThreadCount)
                    .max()
                    .orElse(1);

            for (int i = 0; i <= 10; i++) {
                double value = maxY * i / 10.0;
                int y = y0 - (int) (height * i / 10.0);

                g2.setColor(new Color(220, 220, 220));
                g2.drawLine(x0, y, x0 + width, y);

                g2.setColor(Color.BLACK);
                g2.drawString(String.format(Locale.US, "%.2f", value), 20, y + 5);
            }

            for (int t = 1; t <= maxThreads; t++) {
                int x = x0 + (int) ((t - 1) * (width / (double) Math.max(1,
maxThreads - 1)));

                g2.setColor(new Color(230, 230, 230));
                g2.drawLine(x, y0, x, top);

                g2.setColor(Color.BLACK);
                g2.drawString(String.valueOf(t), x - 5, y0 + 20);
            }

            g2.drawString("Threads", x0 + width / 2 - 20, getHeight() - 25);
            g2.drawString("Speedup", 10, top - 10);

            g2.setColor(Color.RED);
            int yIdeal = y0 - (int) (height * (1.0 / maxY));
            g2.drawLine(x0, yIdeal, x0 + width, yIdeal);
            g2.drawString("Speedup = 1", x0 + width - 90, yIdeal - 5);
```

```
            g2.setColor(new Color(40, 90, 180));

            int prevX = -1;
            int prevY = -1;

            for (PerformancePoint p : points) {
                int x = x0 + (int) ((p.getThreadCount() - 1) * (width / (double)
Math.max(1, maxThreads - 1)));
                int y = y0 - (int) (height * (p.getSpeedup() / maxY));

                if (prevX != -1) {
                    g2.drawLine(prevX, prevY, x, y);
                }

                g2.fillOval(x - 4, y - 4, 8, 8);
                g2.drawString(String.format(Locale.US, "%.2f", p.getSpeedup()), x
- 12, y - 10);

                prevX = x;
                prevY = y;
            }

            g2.setColor(Color.BLACK);
            g2.drawString(String.format(Locale.US, "Sequential time: %.6f ms",
sequentialTimeMs), x0, 25);
        }
    }
}
```

Додаток 3

Код файлу GaussianSolver.java

```
package dev.horbatiuk.pcds.lab1;

public interface GaussianSolver {
    double[] solve(LinearSystem system) throws Exception;
}
```

Додаток 4

Код файлу LinearSystem.java

```
package dev.horbatiuk.pcds.lab1;
import java.util.Arrays;

public class LinearSystem {
    private final int size;
    private final double[][] matrix;
    private final double[] vector;

    public LinearSystem(int size) {
        this.size = size;
        this.matrix = new double[size][size];
        this.vector = new double[size];
    }

    public LinearSystem(double[][] matrix, double[] vector) {
        this.size = matrix.length;
        this.matrix = new double[size][size];
        this.vector = new double[size];
        for (int i = 0; i < size; i++) {
            System.arraycopy(matrix[i], 0, this.matrix[i], 0, size);
        }
        System.arraycopy(vector, 0, this.vector, 0, size);
    }

    public int getSize() {
        return size;
    }

    public double[][] getMatrix() {
        return matrix;
    }

    public double[] getVector() {
        return vector;
    }
    public LinearSystem copy() {
        return new LinearSystem(matrix, vector);
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < size; i++) {
            sb.append(Arrays.toString(matrix[i]))
                    .append(" | ")
                    .append(vector[i])
                    .append(System.lineSeparator());
        }
        return sb.toString();
```

```
    }
}
```

Додаток 5

Код файлу LinearSystemGenerator.java

```
package dev.horbatiuk.pcds.lab1;

import java.util.concurrent.ThreadLocalRandom;

public class LinearSystemGenerator {

    private LinearSystemGenerator() {
    }

    public static LinearSystem generateRandomSystem(int size, int minValue, int
maxValue) {
        LinearSystem system = new LinearSystem(size);
        double[][] A = system.getMatrix();
        double[] b = system.getVector();

        for (int i = 0; i < size; i++) {
            double rowAbsSum = 0.0;

            for (int j = 0; j < size; j++) {
                if (i != j) {
                    A[i][j] = ThreadLocalRandom.current().nextInt(minValue,
maxValue + 1);
                    rowAbsSum += Math.abs(A[i][j]);
                }
            }

            A[i][i] = rowAbsSum + ThreadLocalRandom.current().nextInt(1, 10);
            b[i] = ThreadLocalRandom.current().nextInt(minValue, maxValue + 1);
        }

        return system;
    }

    public static LinearSystem createControlExample() {
        double[][] A = {
                {2, 1, -1},
                {-3, -1, 2},
                {-2, 1, 2}
        };
        double[] b = {8, -11, -3};

        return new LinearSystem(A, b);
    }
}
```

Додаток 6

Код файлу ParallelGaussianSolver.java

```
package dev.horbatiuk.pcds.lab1;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class ParallelGaussianSolver extends SequentialGaussianSolver {

    private static final double EPS = 1e-12;
    private final int threadCount;

    public ParallelGaussianSolver(int threadCount) {
        if (threadCount <= 0) {
            throw new IllegalArgumentException("Кількість потоків має бути більшою
за 0.");
        }
        this.threadCount = threadCount;
    }

    @Override
    public double[] solve(LinearSystem system) {
        double[][] A = system.getMatrix();
        double[] b = system.getVector();
        int n = system.getSize();

        ExecutorService executor = Executors.newFixedThreadPool(threadCount);

        try {
            for (int k = 0; k < n; k++) {
                int pivotRow = findPivotRow(A, k, n);
                swapRows(A, b, k, pivotRow);

                if (Math.abs(A[k][k]) < EPS) {
                    throw new ArithmeticException("Матриця вироджена або майже
вироджена.");
                }

                List<Callable<Void>> tasks = new ArrayList<>();

                for (int i = k + 1; i < n; i++) {
                    final int row = i;
                    final int pivot = k;

                    tasks.add(() -> {
                        double factor = A[row][pivot] / A[pivot][pivot];
                        A[row][pivot] = 0.0;
```

```
                    for (int j = pivot + 1; j < n; j++) {
                        A[row][j] -= factor * A[pivot][j];
                    }

                    b[row] -= factor * b[pivot];
                    return null;
                });
            }

            if (!tasks.isEmpty()) {
                executor.invokeAll(tasks);
            }
        }

        return backSubstitution(A, b, n);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    } finally {
        executor.shutdown();
        try {
            executor.awaitTermination(1, TimeUnit.MINUTES);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
}
}
```

Додаток 7

Код файлу PerformancePoint.java

```
package dev.horbatiuk.pcds.lab1;

public class PerformancePoint {
    private final int threadCount;
    private final double speedup;
    private final double parallelTimeMs;

    public PerformancePoint(int threadCount, double speedup, double
parallelTimeMs) {
        this.threadCount = threadCount;
        this.speedup = speedup;
        this.parallelTimeMs = parallelTimeMs;
    }

    public int getThreadCount() {
        return threadCount;
    }

    public double getSpeedup() {
        return speedup;
    }

    public double getParallelTimeMs() {
        return parallelTimeMs;
    }
}
```

Додаток 8

Код файлу SequentialGaussianSolver.java

```
package dev.horbatiuk.pcds.lab1;

public class SequentialGaussianSolver implements GaussianSolver {

    private static final double EPS = 1e-12;

    @Override
    public double[] solve(LinearSystem system) {
        double[][] A = system.getMatrix();
        double[] b = system.getVector();
        int n = system.getSize();

        for (int k = 0; k < n; k++) {
            int pivotRow = findPivotRow(A, k, n);
            swapRows(A, b, k, pivotRow);

            if (Math.abs(A[k][k]) < EPS) {
                throw new ArithmeticException("Матриця вироджена або майже
вироджена.");
            }

            for (int i = k + 1; i < n; i++) {
                double factor = A[i][k] / A[k][k];
                A[i][k] = 0.0;

                for (int j = k + 1; j < n; j++) {
                    A[i][j] -= factor * A[k][j];
                }

                b[i] -= factor * b[k];
            }
        }

        return backSubstitution(A, b, n);
    }

    protected int findPivotRow(double[][] A, int column, int n) {
        int pivotRow = column;
        for (int i = column + 1; i < n; i++) {
            if (Math.abs(A[i][column]) > Math.abs(A[pivotRow][column])) {
                pivotRow = i;
            }
        }
        return pivotRow;
    }

    protected void swapRows(double[][] A, double[] b, int row1, int row2) {
```

```
        if (row1 == row2) {
            return;
        }

        double[] tempRow = A[row1];
        A[row1] = A[row2];
        A[row2] = tempRow;

        double temp = b[row1];
        b[row1] = b[row2];
        b[row2] = temp;
    }

    protected double[] backSubstitution(double[][] A, double[] b, int n) {
        double[] x = new double[n];

        for (int i = n - 1; i >= 0; i--) {
            double sum = b[i];

            for (int j = i + 1; j < n; j++) {
                sum -= A[i][j] * x[j];
            }

            x[i] = sum / A[i][i];
        }

        return x;
    }
}
```

Додаток 9

Код файлу SolutionResult.java

```
package dev.horbatiuk.pcds.lab1;

import java.util.Locale;

public class SolutionResult {
    private final double[] solution;
    private final long executionTimeNs;

    public SolutionResult(double[] solution, long executionTimeNs) {
        this.solution = solution;
        this.executionTimeNs = executionTimeNs;
    }

    public double[] getSolution() {
        return solution;
    }

    public long getExecutionTimeNs() {
        return executionTimeNs;
    }

    public double getExecutionTimeMs() {
        return executionTimeNs / 1_000_000.0;
    }

    public double getExecutionTimeSec() {
        return executionTimeNs / 1_000_000_000.0;
    }

    public String solutionAsString() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < solution.length; i++) {
            sb.append(String.format(Locale.US, "x[%d] = %.6f%n", i, solution[i]));
        }
        return sb.toString();
    }
}
```

Додаток 10

Код файлу SystemFileManager.java

```
package dev.horbatiuk.pcds.lab1;

import java.io.*;
import java.util.Locale;
import java.util.Scanner;

public class SystemFileManager {

    private SystemFileManager() {
    }

    public static void saveSystemToFile(LinearSystem system, String fileName)
throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(fileName))) {
            int n = system.getSize();
            double[][] A = system.getMatrix();
            double[] b = system.getVector();

            writer.println(n);
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    writer.printf(Locale.US, "%.10f ", A[i][j]);
                }
                writer.printf(Locale.US, "%.10f%n", b[i]);
            }
        }
    }

    public static LinearSystem readSystemFromFile(String fileName) throws
IOException {
        try (Scanner scanner = new Scanner(new File(fileName))) {
            scanner.useLocale(Locale.US);

            int n = scanner.nextInt();
            LinearSystem system = new LinearSystem(n);

            double[][] A = system.getMatrix();
            double[] b = system.getVector();

            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    A[i][j] = scanner.nextDouble();
                }
                b[i] = scanner.nextDouble();
            }

            return system;
        }
```

```
    }

    public static void saveResultsToFile(
            String fileName,
            SolutionResult sequential,
            SolutionResult parallel,
            double speedup
    ) throws IOException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(fileName))) {
            writer.println("Sequential solution:");
            writer.print(sequential.solutionAsString());
            writer.printf(Locale.US, "Sequential time: %.6f ms%n%n",
sequential.getExecutionTimeMs());

            writer.println("Parallel solution:");
            writer.print(parallel.solutionAsString());
            writer.printf(Locale.US, "Parallel time: %.6f ms%n",
parallel.getExecutionTimeMs());
            writer.printf(Locale.US, "Speedup: %.4f%n", speedup);
        }
    }
}
```

Додаток 11

Код файлу Main.java

```
package dev.horbatiuk.pcds.lab1;

import java.util.Locale;
import java.util.Scanner;

public class Main {

    private static final int MAX_ROWS_TO_PRINT = 8;
    private static final int MAX_COLS_TO_PRINT = 8;

    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner scanner = new Scanner(System.in);

        printHeader("Gaussian Elimination Laboratory Work");

        while (true) {
            printMenu();
            int option = readInt(scanner, "Choose option: ");

            if (option == 0) {
                printHeader("Program finished");
                break;
            }

            try {
                if (option == 4) {
                    runChartExperiment(scanner);
                    continue;
                }

                LinearSystem system = switch (option) {
                    case 1 -> createGeneratedSystem(scanner);
                    case 2 -> createSystemFromFile(scanner);
                    case 3 -> LinearSystemGenerator.createControlExample();
                    default -> {
                        printWarning("Invalid option.");
                        yield null;
                    }
                };

                if (system == null) {
                    continue;
                }

                printHeader("Input system");
                printSystemPreview(system);
```

```
                SolutionResult sequentialResult = Benchmark.measure(
                        new SequentialGaussianSolver(),
                        system
                );

                int threads = readInt(scanner, "Enter number of threads: ");

                SolutionResult parallelResult = Benchmark.measure(
                        new ParallelGaussianSolver(threads),
                        system
                );

                double speedup = Benchmark.calculateSpeedup(sequentialResult,
parallelResult);

                printHeader("Computation results");
                printResultBlock("Sequential", sequentialResult);
                printResultBlock("Parallel", parallelResult);
                printPerformanceSummary(sequentialResult, parallelResult,
speedup);

                int saveResults = readInt(scanner, "\nWrite results to file? (1-
yes, 0-no): ");
                if (saveResults == 1) {
                    System.out.print("Enter output file name: ");
                    String fileName = scanner.next();
                    SystemFileManager.saveResultsToFile(fileName,
sequentialResult, parallelResult, speedup);
                    printSuccess("Results saved to file: " + fileName);
                }

            } catch (Exception e) {
                printError("Error: " + e.getMessage());
            }
        }

        scanner.close();
    }

    private static void runChartExperiment(Scanner scanner) throws Exception {
        printHeader("Speedup chart experiment");

        int n = readInt(scanner, "Enter system size n: ");
        int maxThreads = readInt(scanner, "Enter maximum number of threads: ");

        if (maxThreads <= 0) {
            printWarning("Maximum number of threads must be greater than 0.");
            return;
        }
```

```
        LinearSystem system = LinearSystemGenerator.generateRandomSystem(n, -10,
10);

        printHeader("Input system preview");
        printSystemPreview(system);

        SolutionResult sequentialResult = Benchmark.measure(
                new SequentialGaussianSolver(),
                system
        );

        System.out.printf("Sequential baseline time: %.6f ms%n",
sequentialResult.getExecutionTimeMs());
        System.out.println();

        java.util.List<PerformancePoint> points = new java.util.ArrayList<>();

        System.out.println("Threads | Parallel time (ms) | Speedup");
        System.out.println("-------------------------------------");

        for (int threads = 1; threads <= maxThreads; threads++) {
            SolutionResult parallelResult = Benchmark.measure(
                    new ParallelGaussianSolver(threads),
                    system
            );

            double speedup = Benchmark.calculateSpeedup(sequentialResult,
parallelResult);
            points.add(new PerformancePoint(
                    threads,
                    speedup,
                    parallelResult.getExecutionTimeMs()
            ));

            System.out.printf(Locale.US, "%7d | %18.6f | %7.4f%n",
                    threads,
                    parallelResult.getExecutionTimeMs(),
                    speedup
            );
        }

        javax.swing.SwingUtilities.invokeLater(() -> {
            ChartFrame frame = new ChartFrame(points,
sequentialResult.getExecutionTimeMs());
            frame.setVisible(true);
        });
    }

    private static LinearSystem createGeneratedSystem(Scanner scanner) throws
Exception {
        printHeader("Random system generation");
```

```
        int n = readInt(scanner, "Enter system size n: ");
        LinearSystem system = LinearSystemGenerator.generateRandomSystem(n, -10,
10);

        int save = readInt(scanner, "Save generated system to file? (1-yes, 0-no):
");
        if (save == 1) {
            System.out.print("Enter file name: ");
            String fileName = scanner.next();
            SystemFileManager.saveSystemToFile(system, fileName);
            printSuccess("System saved to file: " + fileName);
        }

        return system;
    }

    private static LinearSystem createSystemFromFile(Scanner scanner) throws
Exception {
        printHeader("Read system from file");

        System.out.print("Enter file name: ");
        String fileName = scanner.next();

        LinearSystem system = SystemFileManager.readSystemFromFile(fileName);
        printSuccess("System loaded from file: " + fileName);

        return system;
    }

    private static void printMenu() {
        System.out.println();
        System.out.println("====================================================");
        System.out.println("                      MAIN MENU                     ");
        System.out.println("====================================================");
        System.out.println("1 - Generate random system");
        System.out.println("2 - Read system from file");
        System.out.println("3 - Use control example");
        System.out.println("4 - Build speedup chart");
        System.out.println("0 - Exit");
        System.out.println("----------------------------------------------------");
    }

    private static void printHeader(String title) {
        System.out.println();
        System.out.println("====================================================");
        System.out.println(centerText(title, 50));
        System.out.println("====================================================");
    }

    private static void printSystemPreview(LinearSystem system) {
        int n = system.getSize();
```

```
        double[][] A = system.getMatrix();
        double[] b = system.getVector();

        System.out.printf("System size: %d x %d%n", n, n);

        int rowsToPrint = Math.min(n, MAX_ROWS_TO_PRINT);
        int colsToPrint = Math.min(n, MAX_COLS_TO_PRINT);

        System.out.println();
        for (int i = 0; i < rowsToPrint; i++) {
            System.out.print("│ ");

            for (int j = 0; j < colsToPrint; j++) {
                System.out.printf("%10.3f ", A[i][j]);
            }

            if (colsToPrint < n) {
                System.out.print("... ");
            }

            System.out.printf("│ %10.3f%n", b[i]);
        }

        if (rowsToPrint < n) {
            System.out.println("│        ...
│        ...");
        }
    }

    private static void printResultBlock(String label, SolutionResult result) {
        System.out.println();
        System.out.println("--------------------------------------------------");
        System.out.println(label.toUpperCase() + " SOLUTION");
        System.out.println("--------------------------------------------------");

        double[] solution = result.getSolution();
        int valuesToPrint = Math.min(solution.length, 10);

        for (int i = 0; i < valuesToPrint; i++) {
            System.out.printf("x[%-4d] = %12.6f%n", i, solution[i]);
        }

        if (solution.length > valuesToPrint) {
            System.out.println("...");
            System.out.printf("x[%-4d] = %12.6f%n", solution.length - 1,
solution[solution.length - 1]);
        }

        System.out.printf("%s time: %.6f ms%n", label,
result.getExecutionTimeMs());
    }
```

```
    private static void printPerformanceSummary(
            SolutionResult sequentialResult,
            SolutionResult parallelResult,
            double speedup
    ) {
        double sequentialMs = sequentialResult.getExecutionTimeMs();
        double parallelMs = parallelResult.getExecutionTimeMs();
        double slowdown = parallelMs / sequentialMs;

        System.out.println();
        System.out.println("--------------------------------------------------");
        System.out.println("PERFORMANCE SUMMARY");
        System.out.println("--------------------------------------------------");
        System.out.printf("Sequential time : %12.6f ms%n", sequentialMs);
        System.out.printf("Parallel time   : %12.6f ms%n", parallelMs);
        System.out.printf("Speedup         : %12.4f%n", speedup);

        if (speedup > 1.0) {
            System.out.printf("Result          : parallel version is faster
by %.2f times%n", speedup);
        } else if (Math.abs(speedup - 1.0) < 1e-9) {
            System.out.println("Result          : both versions have nearly
identical speed");
        } else {
            System.out.printf("Result          : parallel version is slower
by %.2f times%n", slowdown);
        }
    }

    private static int readInt(Scanner scanner, String prompt) {
        while (true) {
            System.out.print(prompt);
            if (scanner.hasNextInt()) {
                return scanner.nextInt();
            }
            scanner.next();
            printWarning("Please enter an integer value.");
        }
    }

    private static void printSuccess(String message) {
        System.out.println("[OK] " + message);
    }

    private static void printWarning(String message) {
        System.out.println("[WARN] " + message);
    }

    private static void printError(String message) {
        System.out.println("[ERROR] " + message);
    }
```

```
    private static String centerText(String text, int width) {
        if (text.length() >= width) {
            return text;
        }

        int leftPadding = (width - text.length()) / 2;
        return " ".repeat(leftPadding) + text;
    }
}
```